

Guide Complet : Machine Learning avec Python

1- Introduction au Machine Learning (ML)

Définition :

Le Machine Learning est une branche de l'intelligence artificielle qui permet aux machines d'apprendre à partir des données et de faire des prédictions ou prendre des décisions sans être explicitement programmées.

Nécessité du ML :

- . Traiter de grandes quantités de données
- . Automatiser des décisions
- . Découvrir des patterns complexes difficiles à programmer manuellement

Programmation classique

Machine Learning

Instructions explicites
Code fixe
Résultat prédéfini

Apprentissage à partir des données
Modèle qui s'ajuste automatiquement
Résultat prédit en fonction des données

2- Les données en ML

Rôle :

Les données sont le carburant du ML. La qualité des modèles dépend fortement des données utilisées.

Analyse et exploitation des données :

Exploration statistique et graphique (moyenne, écart-type, corrélation, histogrammes, boxplots...)

Détection des patterns, tendances et anomalies

Prétraitement des données :

- **Nettoyage** : valeurs manquantes, doublons, outliers

- **Encodage** : transformer les catégories en numérique (Label Encoding, One-Hot Encoding)

- **Mise à l'échelle** : Standardisation, Normalisation

- **Sélection des caractéristiques** : corrélations, PCA, importance des features

- **Division du dataset** : train/test (et validation)

Exemple Python :

```
from sklearn.preprocessing import StandardScaler
import pandas as pd
```

```
df = pd.read_csv("data.csv")
df['age'].fillna(df['age'].mean(), inplace=True)
df = pd.get_dummies(df, columns=['ville'])
```

```
scaler = StandardScaler()
df[['taille','poids']] = scaler.fit_transform(df[['taille','poids']])
```

3- Types d'apprentissage

3.1 Apprentissage supervisé

Définition : données étiquetées → modèle apprend à associer entrée → sortie

Exemples : prédiction de prix, classification spam/non spam, reconnaissance d'images

Sous-types :

. **Régression** : prédire une valeur continue (ex: prix d'une maison)

. **Classification** : prédire une catégorie (ex: chat ou chien)

3.2 Apprentissage non supervisé

Définition : données non étiquetées → modèle découvre des structures cachées

Exemples : clustering, réduction de dimension, détection d'anomalies

4- Algorithmes de ML

4.1 Algorithmes de régression

- Linear Regression (Régression linéaire)

Définition : Modèle statistique qui prédit une valeur continue en supposant une relation linéaire entre les variables indépendantes (features) et la variable cible.

Exemple : Prédire le prix d'une maison en fonction de sa surface, nombre de chambres, localisation, etc.

Points clés : Simple, rapide, mais limité si la relation n'est pas linéaire.

- Polynomial Regression (Régression polynomiale)

Définition : Variante de la régression linéaire qui utilise une relation polynomiale entre les variables indépendantes et la cible.

Exemple : Croissance d'une plante selon le temps, où la relation n'est pas strictement linéaire mais suit une courbe.

Points clés : Permet de modéliser des relations non linéaires, mais peut surajuster si le degré du polynôme est trop élevé.

- Random Forest Regressor (Forêt aléatoire pour régression)

Définition : Ensemble d'arbres de décision combinés pour prédire une valeur continue. Chaque arbre prédit indépendamment et la prédiction finale est la moyenne de tous les arbres.

Exemple : Estimer la consommation d'énergie d'une maison selon température, occupation, équipements, etc.

Points clés :

- Réduit le surapprentissage par rapport à un arbre seul

- Gère bien les données complexes et non linéaires

- Réseaux de neurones (Neural Networks pour régression)

Définition : Modèle inspiré du cerveau humain, constitué de couches de neurones interconnectés, capable d'apprendre des relations complexes entre les features et la cible.

Exemple : Prédiction complexe multifeature, comme la valeur d'une maison en prenant en compte les caractéristiques de l'environnement, météo, historique des prix...

Points clés :

Très puissant pour modéliser des relations non linéaires complexes

Nécessite beaucoup de données et un prétraitement approprié

Plus long à entraîner que les modèles classiques

4.2 Algorithmes de classification

- Régression logistique (Logistic Regression)

Définition : Modèle statistique qui prédit la probabilité qu'un exemple appartienne à une classe. La sortie est comprise entre 0 et 1 grâce à la fonction sigmoïde.

Exemple : Prédire si un email est spam ou non.

Sensibilité :

Sensible aux valeurs manquantes → nécessite remplissage ou imputation

Sensible aux variables catégorielles non encodées → nécessite encodage (One-Hot ou Label Encoding)

- Arbre de décision pour la classification (Decision Tree)

Définition : Modèle qui divise récursivement les données en fonction de seuils de caractéristiques pour séparer les classes. Résultat : un arbre de décisions.

Exemple : Classifier des fruits selon taille, couleur et poids.

Sensibilité :

Peu sensible aux valeurs manquantes si la méthode de scikit-learn DecisionTreeClassifier est utilisée, mais mieux vaut imputer.

Peu sensible aux variables catégorielles si elles sont codées en numérique, sinon nécessité d'encodage.

- Forêt aléatoire (Random Forest Classifier)

Définition : Ensemble d'arbres de décision combinés pour améliorer la précision et réduire le surapprentissage.

Exemple : Détection de fraude dans les transactions bancaires.

Sensibilité :

Peu sensible aux valeurs manquantes si imputées

Peu sensible aux variables catégorielles non encodées, mais scikit-learn requiert un encodage numérique

- Machine à vecteur de support (SVM – Support Vector Machine)

Définition : Cherche l'hyperplan qui sépare au mieux les classes en maximisant la marge entre elles. Peut utiliser des kernels pour les données non linéaires.

Exemple : Détection de cancer (malade / sain).

Sensibilité :

Très sensible aux valeurs manquantes → il faut imputer

Très sensible aux variables catégorielles non encodées → encodez-les obligatoirement

Très sensible à l'échelle des variables → standardisation recommandée

- K-Plus Proches Voisins (K-NN)

Définition : Classe un exemple selon la majorité des K voisins les plus proches dans l'espace des features.

Exemple : Reconnaissance d'animaux selon taille et poids.

Sensibilité :

Très sensible aux valeurs manquantes → nécessite imputation

Très sensible aux variables catégorielles non encodées → encodez-les

Très sensible à l'échelle des variables → normalisation ou standardisation nécessaire

- Naive Bayes

Définition : Classifie les données selon le théorème de Bayes, en supposant que les features sont indépendantes.

Exemple : Filtrage spam (probabilité qu'un email soit spam).

Sensibilité :

Sensible aux valeurs manquantes → il faut les traiter

Peu sensible aux variables catégorielles si on utilise la version adaptée (ex: CategoricalNB pour catégoriques)

Encodage numérique requis pour GaussianNB

- Réseau de neurones (Neural Network / MLP Classifier)

Définition : Modèle inspiré du cerveau humain, constitué de couches de neurones qui apprennent des représentations complexes des données pour la classification.

Exemple : Reconnaissance faciale, classification d'images.

Sensibilité :

Très sensible aux valeurs manquantes → imputation obligatoire

Très sensible aux variables catégorielles non encodées → encodage obligatoire

Très sensible à l'échelle des variables → normalisation recommandée

5- Sensibilité des algorithmes au prétraitement

5.1 Algorithmes sensibles à l'échelle

- . KNN, SVM, réseaux de neurones, régression linéaire/logistique
- . Besoin : normalisation ou standardisation, traitement des outliers

5.2 Algorithmes non sensibles à l'échelle

- . Arbres de décision, Random Forest, Gradient Boosting
- . Prétraitement moins critique, mais nettoyage recommandé

6- Optimisation des hyperparamètres

. **Hyperparamètres** : définis avant l'entraînement (ex: nombre de voisins K, profondeur d'arbre)

- . **Objectif** : maximiser performance et généralisation

Techniques :

- 1. Grid Search** → toutes les combinaisons
- 2. Random Search** → combinaison aléatoire
- 3. Optimisation bayésienne** → approche probabiliste, efficace
- 4. Algorithmes évolutionnaires** → recherche inspirée de la sélection naturelle

Exemples Python :

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {'n_estimators': [50,100], 'max_depth':[5,10]}
rf = RandomForestClassifier()
grid_search = GridSearchCV(rf, param_grid, cv=5)
grid_search.fit(X_train, y_train)
print(grid_search.best_params_)
```

7- Validation croisée (Cross-Validation)

Définition : technique pour évaluer un modèle sur plusieurs sous-ensembles du dataset

Types : K-Fold, Stratified K-Fold, Leave-One-Out, Shuffle Split

Avantages :

Évite le surapprentissage

Évalue stabilité du modèle

Compare plusieurs modèles

Exemple Python :

```
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
scores = cross_val_score(model, X, y, cv=5)
print(scores.mean())
```

8- Techniques d'optimisation des paramètres

1. Gradient Descent → descend le gradient pour minimiser la fonction de coût
2. Stochastic / Mini-batch Gradient Descent → pour grands datasets

3. Optimisation adaptative → Adam, RMSProp, Adagrad

4. Méthodes basées sur le second ordre → Newton-Raphson, BFGS

Régularisation pour éviter le surapprentissage :

. L1 (Lasso), L2 (Ridge), Dropout (réseaux de neurones)

9- Sauvegarde du modèle

Pourquoi sauvegarder un modèle ?

Gain de temps : éviter de réentraîner le modèle à chaque fois

Reproductibilité : pouvoir partager le modèle avec d'autres personnes ou systèmes

Déploiement : utiliser le modèle dans une application ou un serveur de production

Formats de sauvegarde

Pickle (.pkl)

Standard Python pour sérialiser des objets

Simple, rapide, mais spécifique à Python

Joblib (.joblib)

Optimisé pour les objets volumineux comme les modèles ML

Plus rapide que Pickle pour les modèles scikit-learn

HDF5 (.h5)

Utilisé pour les réseaux de neurones (Keras/TensorFlow)

Permet de sauvegarder architecture + poids + optimizer

ONNX (.onnx)

Format standard pour partager les modèles entre frameworks (PyTorch, TensorFlow, scikit-learn)

Idéal pour le déploiement inter-plateformes

Bonnes pratiques

- Sauvegarder le modèle et le préprocesseur
 - . Si vous normalisez ou encodez les données, sauvegardez aussi le scaler ou l'encodeur pour transformer les nouvelles données correctement.
- Versionner le modèle
 - . Exemple : model_v1.joblib, model_v2.pkl pour suivre les évolutions
- Test après chargement
 - . Toujours vérifier que le modèle chargé donne les mêmes résultats que celui entraîné
- Déploiement
 - . ONNX ou Pickle/Joblib selon la plateforme cible (Python seulement ou inter-langage)